Beyond Nominal Ratios: Language-Aware Prompt Compression for Multi-hop Question Answering

Thanh-Nhan Truong[1,2] [0009−0007−5012−7578]

[1] Faculty of Information Technology, University of Science, Ho Chi Minh City, Vietnam
25C1505320@student.hcmus.edu.vn
[2] Vietnam National University, Ho Chi Minh City, Vietnam

Abstract. Prompt compression is reported as a ratio: the fraction of the context a compressor keeps. We show that this number is not comparable across languages, and that the gap is large enough to change which method looks cheaper. Measuring token fertility on parallel WMT24++ sentences across four writing systems, a context costs 4.97× more tokens in Hindi, 3.89× in Thai and 2.49× in Vietnamese than its English counterpart under the tokeniser that bills, so a compressor reporting the same nominal ratio in two languages is not delivering the same saving. We propose an *effective ratio* that prices retention in English-equivalent tokens, and show it separates tokenisers that nominal ratios rank as identical (1.02× spread nominal, 1.80× effective). To ground this we reimplement IterCOMP, a state-of-the-art training-free compressor evaluated only on English, and give it its first systematic evaluation on Vietnamese using the complete VimQA benchmark alongside MuSiQue, 2WikiMultiHopQA and HotpotQA ($n \geq 1{,}003$ each, six compression baselines). Two further effects appear only outside English: a boolean-answer normalisation step is worth +8.67 F1 in Vietnamese and exactly 0.0 in English, and undiacriticised text – ubiquitous online – costs every branch about 20 F1 while leaving the ordering intact, the damage falling on the reader rather than on the compressor. We release the implementation and the Vietnamese prompts.

Keywords: Prompt compression · Multi-hop question answering · Tokeniser fertility · Vietnamese NLP · Retrieval-augmented generation

1 Introduction

Multi-hop question answering requires aggregating evidence across documents. Retrieval-augmented generation (RAG) [7] supplies external documents to compensate for a model's bounded knowledge, but introduces two problems: *cost* (input length grows linearly with the number of documents) and *noise* (irrelevant passages degrade reader accuracy).

Corresponding author: Thanh-Nhan Truong, 25C1505320@student.hcmus.edu.vn. Code, prompts (Vietnamese and English), per-dataset results and the extended analyses are at https://github.com/nhantrnh/IterCOMP-X.

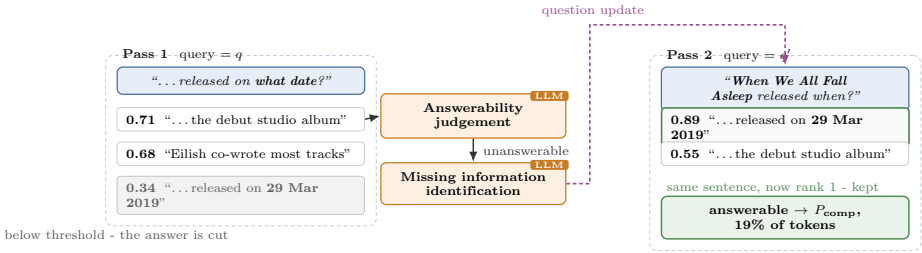


Fig. 1. One VimQA question through two IterCOMP passes. The sentence carrying the release date scores 0.34 and is filtered out, so a single-pass compressor loses the answer; re-scored against the follow-up q' it rises to 0.89 and is kept.

Prompt compression addresses both. However, existing methods [4,10,15,8] compress *once*, conditioned on the original question, and therefore miss the defining property of multi-hop QA: the information needed at a later hop only becomes identifiable after the evidence of the earlier hop is in hand. IterCOMP [17] embeds multi-hop reasoning into the compression loop itself. No public implementation accompanies the original paper, and its evaluation remains restricted to English.

The Vietnamese adaptation and evaluation are our primary contribution, and the reproduction that enables them is reported after it. Five findings follow, four specific to Vietnamese:

- **An effective ratio** that prices retention in English-equivalent tokens. Nominal ratios understate what a Vietnamese context actually costs by 2.39×, because the tokeniser that bills is not the one that counts.
- **A boolean-answer normalisation step** worth +8.67 F1 on Vietnamese (95% CI $[+6.98, +10.47]$) and exactly 0.0 on all three English datasets - a linguistic fix, not a metric-inflating one.
- **Evidence that Vietnamese does not handicap the compressor.** IterCOMP-X reaches 84% of the Oracle ceiling on VimQA; the lower 56% on English MuSiQue follows from that dataset's twenty candidate passages, not from language (Section 5.5).
- **A saturation point for the iteration budget.** Raising `max_iter` from 2 to 5 moves F1 by +1.2 at $n = 500$, inside the noise floor, while retention grows from 0.160 to 0.194 - the extra passes buy context, not accuracy.
- **The first measurement of undiacriticised Vietnamese** under compression. Dropping tone marks costs about 20 F1, but it costs gold context 21.07 and the compressed context 20.26, so the loss belongs to the reader and not to the compressor (Table 7).

We call this configuration IterCOMP-X and stress that it is *not* a new algorithm: the loop is unchanged. It is a deployment configuration plus a metric-level fix, and we report which parts survive significance testing.

None of this is measurable without a working reimplementation; absent a public one, we rebuild it from the paper's description and redesign every prompt. It recovers the central result on all four datasets (Table 5); Section 5.2 compares

our numbers with the original's, and the released repository carries the per-dataset breakdown.

2 Related Work

Prompt compression. Extractive methods keep a subset of tokens or sentences: LLMLingua [4] drops tokens by perplexity; LongLLMLingua [5] adds query awareness; LLMLingua-2 [10] recasts this as token classification, faster and *multilingual* - the decisive property here; Selective-Context [8] removes redundant lexical units. *Abstractive* methods regenerate shorter content, notably RECOMP [15]. All compress in a single pass.

Iterative retrieval. Self-Ask [13] and IRCoT [14] decompose questions and interleave retrieval with reasoning; IterCOMP instead *narrows* the context rather than expanding it. Every one of these is reported on English benchmarks.

Tokeniser disparity. Subword tokenisers split non-English text more finely, so the same content costs more tokens: Ahia et al. [1] price this against commercial APIs and Petrov et al. [11] frame it as unfairness between languages. That literature measures the cost of *text*. What it has not asked is what the disparity does to a *ratio* of two token counts, which is how every compressor above reports its result - the question Section 3.1 takes up.

Vietnamese NLP. The language is well served at the model level - the encoder PhoBERT [9] and the text-to-text ViT5 [12] are both established - and VimQA [6] gives it a multi-hop benchmark built to the HotpotQA format. What is missing is the layer between: to our knowledge no compressor has been evaluated on Vietnamese, so neither the accuracy nor the cost of compressing a Vietnamese context has been measured.

3 Methodology

3.1 The tokeniser tax and the effective ratio

A compression ratio is reported as a fraction of the context kept, and read as a saving. That reading assumes the unit being counted costs the same everywhere. It does not: the tokeniser that bills splits some writing systems far more finely than English, so the same nominal ratio buys different savings in different languages. We measure the gap here and propose a correction.

Four writing systems. The claim above is about Vietnamese, so it needs a control: if every language were billed alike, fertility would not be worth reporting. Measuring 400 parallel WMT24++ pairs per language shows it is not.

Every language exceeds 1.0 with non-overlapping intervals (Fig. 2), so the mismatch is systematic rather than a Vietnamese quirk; Vietnamese at 2.49× is the *mild* case, Hindi being penalised twice as heavily. The 2.49× here (400 pairs) and the 2.39× in Table 1 (930 pairs) are different samples of the same corpus.

Table 1. Token fertility over 930 parallel English-Vietnamese sentence pairs from WMT24++ [3]. All 95% confidence intervals are narrower than ±0.04. Fertility is the mean of per-pair token ratios (Eq. 2), not the ratio of the mean counts in columns two and three; the two differ by up to 0.02 here because pair lengths vary. Fertility depends strongly on the tokeniser: a factor of two separates `cl100k_base` from XLM-R. [a]GPT-4, GPT-3.5-turbo; [b]GPT-4o. Which tokeniser bills depends on the served model, so exposure to fertility is set by that choice as much as by the language.

Tokeniser	Vi tok	En tok	Fertility
`cl100k_base`[a]	105.6	44.1	**2.39**×
`o200k_base`[b]	63.3	43.3	1.49×
XLM-R (used by bge-m3)	57.9	49.7	1.19×
`\w+` (unit of "ratio")	47.6	35.7	1.34×

Crucially, fertility depends on the tokeniser measured. The compression ratio (Table 6) counts with `\w+`, where fertility is only 1.34×, so comparing ratios across languages is close to fair; but *API cost* is billed on the served model's tokeniser - 2.39× under `cl100k_base` against 1.49× under `o200k_base` - while bge-m3's XLM-R is only 1.19×. On the 200 VimQA contexts here a context averages 1,110 `\w+` words but 2,364 `cl100k_base` tokens, so our ratios *understate* billing 2×.

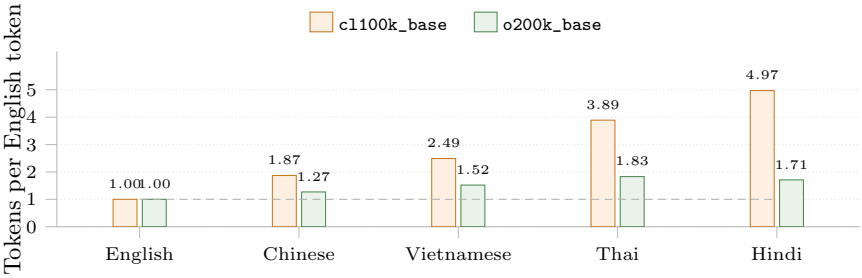


Fig. 2. What the same content costs in tokens, relative to its English translation, over 400 WMT24++ pairs per language. English is 1.0 by construction (dashed). Under `cl100k_base` a Hindi context costs almost five times its English counterpart, so two systems reporting the same nominal ratio in Hindi and in English are not delivering the same saving. Changing the tokeniser changes the tax more than changing the language does: `o200k_base` flattens every bar, and reorders Thai above Hindi.

The effective ratio. Write $|x|_{\mathcal{T}}$ for the number of tokens the billed tokeniser $\mathcal{T}$ assigns to a string x, and let D be the retrieved documents and P_{comp} the compressed prompt. The quantity compression papers report is

$$\text{Ratio}_{\text{nom}} = \frac{|P_{\text{comp}}|_{\mathcal{T}}}{|D|_{\mathcal{T}}}. \tag{1}$$

Let $\phi_{\mathcal{T}}(\mathcal{L})$ be the fertility of language $\mathcal{L}$ under $\mathcal{T}$, measured over a parallel corpus $\mathcal{D}_{\text{par}}$ of aligned pairs as the mean of the per-pair ratios

$$\phi_{\mathcal{T}}(\mathcal{L}) \;=\; \frac{1}{|\mathcal{D}_{\text{par}}|} \sum_{(x_{\mathcal{L}},x_{\text{en}})\in\mathcal{D}_{\text{par}}} \frac{|x_{\mathcal{L}}|_{\mathcal{T}}}{|x_{\text{en}}|_{\mathcal{T}}}, \tag{2}$$

which is what Table 1 and Fig. 2 report. We define the *effective ratio* as what the compressed context costs against an English original of the same content,

$$\text{Ratio}_{\text{eff}} \;=\; \text{Ratio}_{\text{nom}} \cdot \phi_{\mathcal{T}}(\mathcal{L}), \tag{3}$$

so that (3) gives $\text{Ratio}_{\text{eff}} = \text{Ratio}_{\text{nom}}$ for English, where $\phi = 1$, and rises above the nominal figure exactly as far as the language is penalised by the tokeniser. Keeping a fifth of a Vietnamese context is not the saving that keeping a fifth of an English one is, and the second number says by how much. The correction is deliberately a single factor: fertility belongs to the language-tokeniser pair rather than to the compressor, so it is applied to the retention figure rather than charged against the compressor's score. Reporting the pair $(\text{Ratio}_{\text{nom}}, \text{Ratio}_{\text{eff}})$ costs one extra column and leaves the familiar number intact.

Does the effective ratio discriminate? A second metric that moved with the first would add nothing. Holding the compressed contexts fixed - the same 50 VimQA questions, the same IterCOMP output, only the tokeniser changed - the nominal ratio barely moves while the effective ratio does:

Table 2. Nominal against effective ratio for one fixed set of compressed contexts (50 VimQA questions, same IterCOMP-X output, tokeniser varied).

Tokeniser	nominal	effective
`cl100k_base`	0.236	**0.564**
`o200k_base`	0.240	0.357
`\w+`	0.234	**0.314**
spread	1.02×	**1.80**×

Because the contexts are byte-identical across the three rows, the whole of that 1.80× is the tokeniser's and none of it the compressor's. A nominal ratio reported on its own therefore conceals all of it, which is the case for adding the second column rather than replacing the first.

3.2 Iterative multi-hop compression

Given a question q and document set D, we seek a compressed context P_{comp} with $|P_{\text{comp}}| \ll |D|$ such that the reader still answers correctly. IterCOMP is training-free: all parameters stay frozen. Each document is decomposed into sentence-level evidence segments $e_{i,j}$. Figure 1 traces one question through two iterations.

Relevant evidence filtering. The *semantic* signal is the inner product of dense representations from a text encoder $E(\cdot)$:

$$S_{\text{sem}}(q, e_{i,j}) = E(q)^{\top} E(e_{i,j}). \tag{4}$$

This can miss exact lexical cues. Following M3-Embedding [2], the importance weight of token t in a sequence $u \in \{q, e_{i,j}\}$ is

$$w_t^u = \mathrm{ReLU}(\mathbf{w}_{\mathrm{lex}}^\top E_t(u)), \tag{5}$$

where $\mathbf{w}_{\mathrm{lex}}$ is a projection vector and $E_t(u)$ the encoder's contextual state at t. The *lexical* score is a sparse inner product over co-occurring tokens:

$$S_{\mathrm{lex}}(q, e_{i,j}) = \sum_{t \in q \cap e_{i,j}} w_t^q \cdot w_t^{e_{i,j}}. \tag{6}$$

The two are combined into a dual-aspect relevance score:

$$S_{\mathrm{dual}} = \lambda\, S_{\mathrm{sem}} + (1 - \lambda)\, S_{\mathrm{lex}}. \tag{7}$$

Filtering uses a percentile threshold rather than a fixed top-k. Writing $\mathcal{E} = \{e_{i,j}\}$ for the segments of every document in D and $\mathcal{S} = \{S_{\mathrm{dual}}(q, e) \mid e \in \mathcal{E}\}$ for their scores,

$$E_{\mathrm{cand}} = \{e \in \mathcal{E} \mid S_{\mathrm{dual}}(q, e) \geq \mathrm{Percentile}(\mathcal{S}, k)\}. \tag{8}$$

The threshold therefore *adapts* to each question's score distribution rather than committing to a fixed budget.

Answerability judgement and follow-up generation. These two decision points are what make IterCOMP "reasoning-aware": the query evolves with the evidence state instead of staying fixed at q. In step 2 the reader is shown the evidence accumulated so far and asked whether it suffices to answer q, replying with a single token. The reply is read strictly: anything other than `ANSWERABLE_YES` continues the loop, so a malformed answer costs an extra pass rather than a wrong stop. When the evidence is judged insufficient, step 3 asks for one short follow-up question naming the information still missing, and that follow-up becomes q' for the next pass. The loop ends at whichever comes first, a sufficient verdict or `max_iter`. Segments accumulate rather than replace: from $E_{\mathrm{accum}} \leftarrow \emptyset$, each pass m applies $E_{\mathrm{accum}} \leftarrow E_{\mathrm{accum}} \cup E_{\mathrm{cand}}^{(m)}$; on exit the union is deduplicated and restored to source-document order, so P_{comp} reads as narrative rather than as a relevance ranking - which matters when the reader must chain hops across it. Neither prompt is published in the original paper; ours are released in full, in Vietnamese and English.

4 Experimental Setup

4.1 Datasets

The three English datasets are standard for multi-hop QA and are exactly those used by the original paper. VimQA [6] is a Vietnamese multi-hop QA dataset built from Vietnamese Wikipedia, annotated with reasoning type (bridge / comparison) and gold supporting sentences; crucially, it uses the same format as HotpotQA, so our code runs on Vietnamese essentially unchanged. We use the distractor setting for all four, giving 10-20 candidate passages per question, so

Table 3. Development-set sizes. #Q is the full split and #eval the sample actually run; VimQA is evaluated in full.

Dataset	Lang.	#Q	#eval	#psg
HotpotQA (distractor)	En	7,405	1,500	10
2WikiMultiHopQA	En	12,576	1,500	10
MuSiQue-Ans	En	2,417	1,500	20
VimQA	**Vi**	**1,003**	**1,003**	**10**

Table 4. Hyperparameters from the original paper, which used a LLaMA-3-8B reader; VimQA, absent there, uses $k = 90$ as HotpotQA. The reader is 4-bit NF4 (bitsandbytes, double quantisation) under HuggingFace Transformers, decoding greedily at most 48 new tokens, so no sampling variance enters the numbers.

Parameter	Value
Max iterations	5
λ	0.6
k (percentile)	90; 85 for 2Wiki
Encoder	bge-m3 (frozen)
Reader	Qwen2.5-7B-4bit

no Wikipedia index is required and cost is per question rather than per corpus. Because MuSiQue withholds test answers, we report on development sets throughout, and all four datasets are evaluated under a single reader (Qwen2.5-7B at 4-bit) so the numbers compare like with like; Table 3 gives the statistics.

We adopt the original paper's hyperparameters (Table 4).

4.2 Baselines

Raw Document concatenates everything with no compression. Oracle keeps gold supporting sentences only - the upper bound for any document-level method, and a diagnostic: if Oracle is also low, the bottleneck is the reader rather than the compressor. LLMLingua-2 [10] is our main compression baseline, chosen because its token classifier is natively multilingual and covers Vietnamese, so it meets Vietnamese on equal terms and can run on all four datasets.

Five more run on VimQA at $n = 200$: LLMLingua [4], LongLLMLingua [5], Selective-Context [8], RECOMP [15] in both forms, and CompAct [16]. Each carries an English-only component, so they form a wider field rather than like-for-like comparisons and stay out of the main table. R2C we could not run: no public checkpoint.

4.3 Metrics

We compute Exact Match and token-level F1 under the standard HotpotQA normalisation, plus the nominal compression ratio. Tables report F1, the less brittle of the two on free-form answers; EM follows the same ordering and is in the released per-question records. Nominal retention is counted in `\w+` word tokens, the unit the original protocol uses; Eq. 3 converts it to billed cost. That normalisation is built for English, so for Vietnamese we preserve diacritics - stripping

Table 5. Main results: F1* - F1 with the question-guided boolean normalisation of Section 5.5, which is inert on English - under Qwen2.5-7B at 4-bit; the second row gives n. The last two rows are IterCOMP-X minus that baseline. † paired bootstrap interval excludes zero. Bold marks whichever of Raw and IterCOMP-X is higher. Per-dataset discussion is in the released repository.

	VimQA 1003	MuSiQue 1500	2Wiki 1500	HotpotQA 1500
Oracle	61.03	44.90	57.65	71.40
Raw	46.87	21.16	35.72	**59.24**
IterCOMP-X	**51.29**	**25.35**	**36.72**	54.58
LLML-2	35.31	14.63	19.24	36.00
Δ vs. Raw	$+4.42^{\dagger}$	$+4.19^{\dagger}$	+1.00	$-4.66^{\dagger}$
Δ vs. LLML-2	$+15.98^{\dagger}$	$+10.72^{\dagger}$	$+17.48^{\dagger}$	$+18.58^{\dagger}$

them collapses distinct words (*má* / *mà*) and renders EM/F1 meaningless. The released code documents this and the other text-handling prerequisites.

5 Results and Discussion

5.1 Main results across four datasets

The central claim holds on all four. Fig. 3 plots accuracy against context retained; sentence-level compression beats token-level LLMLingua-2 by +10.72 to +18.58 F1 (Table 5, last row), every paired bootstrap interval clear of zero, while keeping 0.194 to 0.354 of the context. The two are not matched on cost: LLMLingua-2 classifies tokens once with a small encoder, IterCOMP-X calls a 7B model twice per pass, 2.6 to 7.4× the time per question (Section 5.3). The margin says sentence-level selection keeps multi-hop evidence that token-level scoring discards - not that it is cheaper. This is the one result independent of dataset, reader and sample size. At $n = 200$ on VimQA it also leads all six baselines, by +11.36 F1 over LLMLingua-2, CI $[+4.93, +17.84]$.

The full ordering does not reproduce uniformly. The paper reports Oracle > IterCOMP > Raw > LLMLingua-2. We reproduce it on VimQA (IterCOMP-X over Raw, +4.42, interval clear) and on MuSiQue (+4.19; at $n = 500$ the same margin read +3.31 and crossed zero, so tripling the sample decided it); the order matches on 2Wiki, where +1.00 does not separate; and on HotpotQA **Raw overtakes IterCOMP-X** by 4.66 F1, interval clear. The four gaps are therefore +4.42, +4.19, +1.00 and −4.66 F1; Section 5.4 accounts for the reversal.

Small samples lacked power rather than validity. At $n = 50$ those gaps fell inside the paired detection threshold of 18.0 F1 on both datasets, so we held the accuracy claim until the full VimQA run settled it. The reading that the margin grows with task complexity does not survive either: +15.98 on VimQA against +10.72 on MuSiQue.

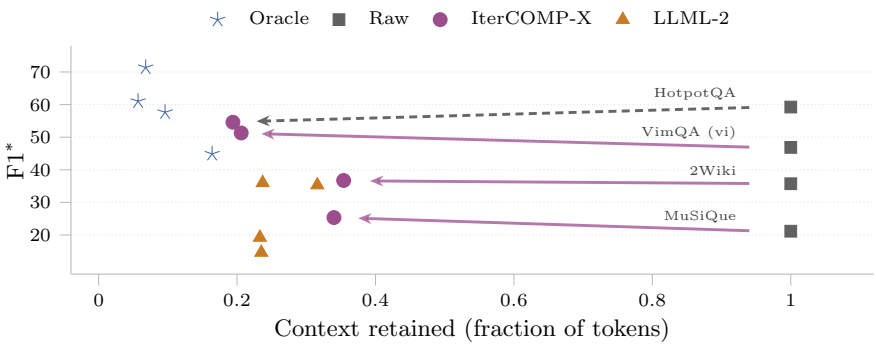


Fig. 3. Accuracy against context retained, all four datasets. Up and to the left is better. Each arrow runs from a dataset's Raw point to its IterCOMP-X point: on VimQA, MuSiQue and 2Wiki the arrow rises while moving left, so compression buys accuracy *and* tokens. On HotpotQA (dashed) it falls: the reader handles the full context well enough that filtering costs 4.66 F1, and only the token saving survives. Oracle marks the ceiling perfect selection would reach.

Table 6. F1 from the original paper (LLaMA-3-8B reader) against our reproduction (Qwen2.5-7B at 4-bit, $n = 50$), plus our Vietnamese results. VimQA has no published baseline; the paper does not evaluate it.

Method	MuSiQue		VimQA
	paper	ours	ours
Oracle	37.51	57.0	53.3
Raw Documents	19.92	32.8	43.8
LLMLingua-2	14.72	13.5	30.7
IterCOMP	27.36	35.1	43.0
ratio	*0.14*	*0.32*	*0.19*
Oracle/Raw	*1.88*	*1.74*	*1.22*

5.2 Comparison with the original paper

This table is at $n = 50$, the paper's operating point, so its MuSiQue figures sit above the full-scale ones (35.1 against 25.35): the first fifty questions are simply easier. Absolute numbers are not comparable across readers, but two invariants indicate a faithful reimplementation. First, the **method ordering reproduces** - Oracle > IterCOMP > Raw > LLMLingua-2 on MuSiQue at both sample sizes, and on 2Wiki at scale (Section 5.1). Second, the Oracle/Raw ratio is in range: 1.74 at $n = 50$ and 2.12 at $n = 1{,}500$ against the paper's 1.88. That ratio is the headroom perfect selection buys over full context. It is a property of the task, not of the reader, so its agreement confirms that decomposition, gold labels and F1 are all correct.

Matching the operating point. At $n = 500$ on MuSiQue IterCOMP-X sits 0.62 F1 *below* the paper's (26.74 against 27.36) while Oracle and Raw, which our compression code never touches, sit far above it (+12.33 Oracle, +3.51 Raw) on

the reader alone. That −0.62 compares systems at different operating points, 0.349 retention against 0.14; sweeping k to match compression puts $k = 96$ at 0.160 retention and 29.04 F1, +1.68 over the paper.

5.3 What the compression costs to run

The saving is in tokens read, not in time. Two extra LLM calls per pass raise wall-clock per question even as the context shrinks: 2.88 to 4.52 s on VimQA and 3.12 to 10.90 s on MuSiQue, against contexts cut to 21% and 35% of Raw. The loop is not equally long on the two - 1.58 passes on VimQA against 3.14 on MuSiQue, whose twenty distractor paragraphs keep the answerability judge unsatisfied. LLMLingua-2, a single encoder pass, costs 1.50 and 1.47 s: faster than Raw everywhere, and the 2.6×–7.4× gap to IterCOMP-X of Section 5.1.

Which matters depends on where the model runs. Behind a per-token API, reading a fifth of the context is the bill and the extra passes are the vendor's problem. Self-hosted the arithmetic inverts: the GPU is the budget, and tripling latency to save context is a poor trade unless the context would not have fitted.

5.4 The ordering is reader-dependent

Oracle exceeds Raw on every dataset while reading 5.7% to 16.4% of the tokens, confirming the premise the literature rests on: irrelevant passages *do* degrade accuracy. But the size of that gap decides whether compression can exploit it. Oracle/Raw is 2.12 on MuSiQue and 1.30 on VimQA, where IterCOMP holds its own, but only 1.21 on HotpotQA, where Raw wins outright. **The premise is reader-dependent**: a reader that already copes with distractors leaves compression little to recover. Swapping our 7B reader for Qwen2.5-0.5B on HotpotQA, the compression pipeline fixed, restores IterCOMP-X to +7.66 F1 over Raw (37.88 against 30.22) - reversing the −4.66 the 7B reader gives, and recovering the ordering the original paper reports with its own weaker reader. This check runs at $n = 500$ rather than the 1,500 of Table 5: a second full-scale sweep was outside our compute budget, and the reversal is far larger than the sampling noise at that size. The HotpotQA reversal is a property of the reader, not a defect of the compressor.

5.5 Vietnamese-specific findings

Running the method on Vietnamese surfaces four results. Boolean answer normalisation is worth +8.67 F1 on VimQA at $n = 1{,}003$, interval $[+6.98, +10.47]$, and exactly 0.0 on each of MuSiQue, 2Wiki and HotpotQA, isolating a language-specific fix from a metric-inflating one: a reader asked a Vietnamese question often answers "Yes", which is correct and scores 0 against the gold "đúng". The step post-processes the reader's output and is gated by the *question* - it fires on the closing *phải không / đúng không* frame, never on the gold answer, so it is available at inference. An unambiguous affirmative or negative in the prediction is rewritten to the dataset's own label for that polarity; anything matching neither or both is left alone, and a wrong-polarity prediction maps to the opposite label and still scores zero. The lexicons are released with the code. Nominal compression ratios understate token cost on Vietnamese by 2.39× because the tokeniser that bills is not the one that counts; we propose an *effective ratio* that

folds fertility back in. Vietnamese does not handicap the compressor: IterCOMP-X recovers 84% of the Oracle ceiling on VimQA (51.29 against 61.03) but only 56% on MuSiQue (25.35 against 44.90). The gap tracks distractor density rather than language: VimQA offers ten candidate passages per question and MuSiQue twenty, so perfect selection is worth more on MuSiQue and a compressor has further to fall short of it.

Undiacriticised Vietnamese. Vietnamese is often typed without tone marks online, a setting no compression paper evaluates. Stripping diacritics is expensive: *má*, *mà* and *mã* collapse into one string, 20.8% of VimQA's syllables merge, and both branches lose about 20 F1 (Table 7). It does not change the ordering: gold leads by 4.81 with tone marks and by 3.99 and 4.05 without, no interval separating. The damage is the reader's: gold context loses 21.07 F1 against the compressed branch's 20.26, so compression degrades 0.81 F1 *less* than perfect evidence does and adds no extraction error of its own.

One point matters for anyone repeating this: supporting facts address documents by title, so diacritics must come off the annotations as well as the context. Strip one side only and the match breaks, leaving the gold branch empty.

Table 7. Undiacriticised Vietnamese ($n = 200$, 7B reader). Gold context leads in every setting and none of the three gaps separates: the paired bootstrap intervals are $[-10.93, +1.44]$, $[-9.51, +1.50]$ and $[-9.48, +1.48]$. Stripping tone marks costs both branches about 20 F1, and costs them equally. `strip` leaves the gold answer accented, so the reader must restore marks it never saw; `both` strips the reference too, scoring entity recovery rather than diacritics.

Setting	IterCOMP-X	Gold	Δ
`keep`	50.17	54.97	−4.81
`strip`	29.91	33.90	−3.99
`both`	30.25	34.30	−4.05

6 Error Analysis

We hand-categorised 50 IterCOMP predictions on VimQA (Table 8). Most errors belong to the *reader*: Oracle sees the gold evidence yet still reaches only 61.03 F1. Of IterCOMP's 48.7 F1 of error, 39.0 - **80%** - survives even perfect retrieval, so no compressor can address it.

Table 8. IterCOMP-X error distribution on VimQA ($n = 50$) under two readers. Format errors fall from 32% to 10% with the larger reader, a reader limitation rather than a compression failure.

Category	0.5B	7B
Fully correct (F1 = 1)	8%	16%
Partially correct (0 < F1 < 1)	34%	32%
Format error on yes/no	**32%**	**10%**
Fully wrong (F1 = 0)	26%	42%

With the 7B reader, **10%** of errors are not compression failures at all: yes/no questions answered correctly in substance but in the wrong surface form (F1 of 0). That bucket was **32%** under a 0.5B reader - "tức là yes" for "đúng", a small model code-switching into English - and falls on the identical prompt as the reader grows, so boolean answers need language-aware normalisation, released with the code, rather than a compression fix. Partially correct answers give a related but wrong entity inside a correctly retained segment; fully wrong ones are mostly compositional, where the Raw-Oracle gap is widest, so distractors rather than the compressor drive them.

A further **9.8% of VimQA is a single question template** - *"X lớn tuổi hơn Y phải không?"* and its negation *nhỏ tuổi hơn*, "is X older than Y?" - on which 44.9% of answers score zero against 37.1% elsewhere, because the task is arithmetic over two retrieved birth dates rather than retrieval. Under a 0.5B reader it collapses to 92.1% zeros.

7 Limitations and Ethical Considerations

Limitations. Absolute numbers are not comparable to the paper's: no prompts were released, the reader differs (Qwen2.5-7B at 4-bit against LLaMA-3-8B), and our retention is 0.349 against their 0.14. Every headline number comes from one reader; greedy decoding rules out sampling noise but not a quantisation artefact, and Section 5.4's 0.5B run is our only check. The sharpest limit is that fertility is measured on four writing systems while compression is run on two, so the effective ratio is argued generally and shown on one language pair. Hindi, the worst case, is where it would matter most and is what we could not test: no Hindi or Thai multi-hop benchmark carries the sentence-level supporting facts the Oracle branch needs.

Ethics. VimQA's CC BY-NC-SA 4.0 licence confines us to research use, and compression is lossy: dropping a negation leaves fluency intact while inverting the answer, so sensitive deployments should verify against the source. A fix worth 8.67 F1 in Vietnamese and 0.0 F1 in English further cautions that "multilingual" models are not equally reliable across languages.

8 Conclusion

A compression ratio is not comparable across languages. The same content costs 4.97× more tokens in Hindi and 2.49× more in Vietnamese under the tokeniser that bills, so an uncorrected ratio overstates the saving by 1.80× across tokenisers it treats as identical. The effective ratio is one extra column and fixes that.

We reproduced IterCOMP without its code and gave it its first Vietnamese evaluation, released as IterCOMP-X. Sentence-level beats token-level compression on all four datasets, but the ordering is reader-dependent: a stronger reader gains enough from full context that the advantage narrows or reverses, leaving a saving in tokens rather than in time. Neither Vietnamese nor the loss of its tone marks handicaps the compressor; the reader does, failing on ∼30% of questions even given gold evidence.

Acknowledgments. Supported by research funding from the Faculty of Information Technology, University of Science, VNU-HCM.

References

1. Ahia, O., Kumar, S., Gonen, H., Kasai, J., Mortensen, D., Smith, N.A., Tsvetkov, Y.: Do all languages cost the same? Tokenization in the era of commercial language models. In: Proceedings of EMNLP (2023), https://aclanthology.org/2023.emnlp-main.614/
2. Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., Liu, Z.: M3-Embedding: Multilinguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In: Findings of ACL (2024), https://aclanthology.org/2024.findings-acl.137/
3. Deutsch, D., Briakou, E., Caswell, I., Finkelstein, M., Galilee, R., Juraska, J., Kovacs, G., Lee, A., Rudder, J., Freitag, M.: WMT24++: Expanding the language coverage of WMT24 to 55 languages and dialects. In: Proceedings of WMT (2025), https://huggingface.co/datasets/google/wmt24pp
4. Jiang, H., Wu, Q., Lin, C.Y., Yang, Y., Qiu, L.: LLMLingua: Compressing prompts for accelerated inference of large language models. In: Proceedings of EMNLP (2023), https://aclanthology.org/2023.emnlp-main.825/
5. Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.Y., Yang, Y., Qiu, L.: LongLLMLingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In: Proceedings of ACL (2024), https://aclanthology.org/2024.acl-long.91/
6. Le, K., Nguyen, H., Le Thanh, T., Nguyen, M.: VIMQA: A Vietnamese dataset for advanced reasoning and explainable multi-hop question answering. In: Proceedings of LREC (2022), https://aclanthology.org/2022.lrec-1.700/
7. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Proceedings of NeurIPS (2020)
8. Li, Y., Dong, B., Guerin, F., Lin, C.: Compressing context to enhance inference efficiency of large language models. In: Proceedings of EMNLP (2023), https://aclanthology.org/2023.emnlp-main.391/
9. Nguyen, D.Q., Nguyen, A.T.: PhoBERT: Pre-trained language models for Vietnamese. In: Findings of EMNLP (2020), https://aclanthology.org/2020.findings-emnlp.92/
10. Pan, Z., Wu, Q., Jiang, H., Xia, M., Luo, X., Zhang, J., Lin, Q., Rühle, V., Yang, Y., Lin, C.Y., Zhao, H.V., Qiu, L., Zhang, D.: LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In: Findings of ACL (2024), https://aclanthology.org/2024.findings-acl.57/
11. Petrov, A., La Malfa, E., Torr, P., Bibi, A.: Language model tokenizers introduce unfairness between languages. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
12. Phan, L., Tran, H., Nguyen, H., Trinh, T.H.: ViT5: Pretrained text-to-text transformer for Vietnamese language generation. In: Proceedings of NAACL Student Research Workshop (2022), https://aclanthology.org/2022.naacl-srw.18/
13. Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N.A., Lewis, M.: Measuring and narrowing the compositionality gap in language models. In: Findings of EMNLP (2023), https://aclanthology.org/2023.findings-emnlp.378/

14. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In: Proceedings of ACL (2023), https://aclanthology.org/2023.acl-long.557/
15. Xu, F., Shi, W., Choi, E.: RECOMP: Improving retrieval-augmented lms with context compression and selective augmentation. In: Proceedings of ICLR (2024), https://openreview.net/forum?id=mlJLVigNHp
16. Yoon, C., Lee, T., Hwang, H., Jeong, M., Kang, J.: CompAct: Compressing retrieved documents actively for question answering. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (2024)
17. Yun, J., Kim, Y.: IterCOMP: Reasoning-aware adaptive prompt compression for multi-hop question answering. In: Proceedings of ACL (Volume 1: Long Papers). San Diego, California (2026), https://aclanthology.org/2026.acl-long.1559/